Automated Bootstrapping Placement for Encrypted Inference

CS5491 Artificial Intelligence Project Milestone

Member 1: Xing Chunjie | Student ID: 72655910
Member 2: Yang Mianji | Student ID: 72655251
Member 3: Zhang Qi | Student ID: 72655380
Contact email: 72655910@cityu-dg.edu.cn

1. Problem and approach

Encrypted computation has a limited capacity for further operations. In the course model, this capacity is represented by an integer ciphertext level. Bootstrapping restores levels when later operations would otherwise lack enough capacity, but it adds substantial latency. Our task is to place these refreshes in the supplied multilayer perceptron (MLP) and attention graphs, including their LayerNorm and Softmax operations, so that every operation has enough levels and total estimated inference latency is minimised.

The choices interact. One refresh can serve several consumers of the same producer, and changing levels also changes ordinary operator costs. A placement with fewer refreshes can therefore cost more overall. We account for these interactions by choosing levels and refreshes jointly, reconstructing the resulting computation graph, then checking its legality and full cost before comparing it with the instructor's placement.

We use OR-Tools CP-SAT, a solver for integer and logical constraints, because the finite levels, discrete insertion choices and shared costs can be expressed together. The instructor supplies the graphs, latency functions and scorer; OR-Tools supplies the search engine. We implemented the formulation, reconstruction and checking workflow described below.

2. Choosing legal placements

Our model tracks the level entering and leaving each operator. For operator v, input level a_v and level consumption d_v give output level b_v = a_v - d_v; the input must lie in that operator's valid range. For each original edge, we choose whether to refresh and what level to deliver. Delivery cannot exceed the available level after any refresh; a lower delivery requires a Modswitch, which lowers levels at zero modelled cost. At a merge, the receiving operator uses the minimum delivered predecessor level. The chosen graph input is fixed, and no final output level is required.

Two refresh types are available. Bootstrap1 (B1) restores level 14 and costs 1040.15 ms per ciphertext. Bootstrap2 (B2) restores level 13 and costs 1293.13 ms per pair, and is allowed only for tensors of 8 or 16 ciphertexts. We activate each producer's refresh once for each type used by its consumers and charge it once, even when several edges use it. Both types may serve different consumers of one producer. The objective adds these shared refresh costs to all ordinary operator costs.

The validated instructor placement gives the search a feasible cost ceiling and hints, without excluding a better solution. CP-SAT also reports a lower bound on achievable cost. When a valid placement's cost matches this bound, it establishes optimality within the encoded model.

3. Reconstructing and checking the graph

The model's choices must become a computation that obeys the original graph's rules. We reconstruct a directed acyclic graph (DAG) that preserves the computation: removing the inserted chains recovers all original operators, edges and ciphertext counts. An original edge may remain unchanged, receive only a Modswitch, or receive B1 or B2 followed by an optional Modswitch. The chosen refresh and delivered level determine this chain. A producer's refresh is shared by type; identical switches are shared when their producer, incoming refresh mode and target level match. We insert switches only for strict level decreases, so shared operations are represented and charged once.

We check structure and levels separately from the score: an inexpensive graph is useful only if it still performs the permitted computation. For every saved placement, our audit checks preservation of the original graph, allowed insertions, shared prefixes, integer levels, operator input ranges, merges, ciphertext counts and B2 eligibility. We independently sum costs with exact decimal arithmetic and compare them with the unchanged instructor scorer. Integer scaling preserves the supplied four-decimal costs in the model; numerical bounds, tolerances and implementation details are recorded in the accompanying evidence index.

The validation suite passed 44 tests, including failure paths and falsified summaries. We also exhaustively enumerated six tiny graphs, obtaining agreement with hand arithmetic, CP-SAT, reconstruction, audit and scoring. These examples check the modelling and reconstruction on cases small enough to inspect completely. Enumeration bypasses CP-SAT and reconstruction, but shares a rules table with modelling and audit, and original-graph checks with model building. Agreement therefore provides a cross-check of implementation within those shared assumptions, rather than an independent interpretation of the requirements or another full-scale benchmark.

4. Comparison design

We ask whether our placements reduce the same computation's estimated latency relative to the instructor's fixed placement. We calibrated all 30 baselines and compared each MLP or attention placement at the same input level, covering levels 0 through 14 with maximum level 14. Every comparison includes ordinary operators and distinct shared insertions. This is two fixed graphs under 15 input conditions each.

The saved comparison ran on 28 September 2026 with one CPU search worker, seed 5491 and a 60-second limit per condition. Exact software versions, source revision, settings and saved graphs are linked in the version README. We separate estimated inference latency from measured solve time and process memory: the baseline performs no placement search, so its scoring time is not another optimiser's search time. These measurements are preliminary evidence; the final protocol and independent reproduction remain outstanding.

5. Preliminary results

All 30 saved placements passed structural and level checks and reduced their same-input baseline costs. Their objectives matched solver lower bounds, establishing optimality within the encoded course model. No condition failed, timed out or used a baseline fallback. Table 1 groups inputs with equal optimised costs; Figure 1 shows how those costs compare with the baseline across all input levels. Both reveal a lower-cost range once the starting level is high enough, which we explain in Section 6.

Block, inputs	Latency (ms)	Reduction (%)	B1 / B2
MLP, 0-13	26069.9524	7.3223-7.3291	4 / 4
MLP, 14	21911.5924	22.1113	0 / 4
Attention, 0-5	45698.2664	14.6755	10 / 8
Attention, 6-14	41537.6664	22.4438	6 / 8

Table 1. Estimated latency and reduction for all 30 conditions. B1 counts ciphertexts and B2 counts pairs; shared operations count once. Their sum is the course bootstrap count, not the number of refresh nodes drawn in a graph.

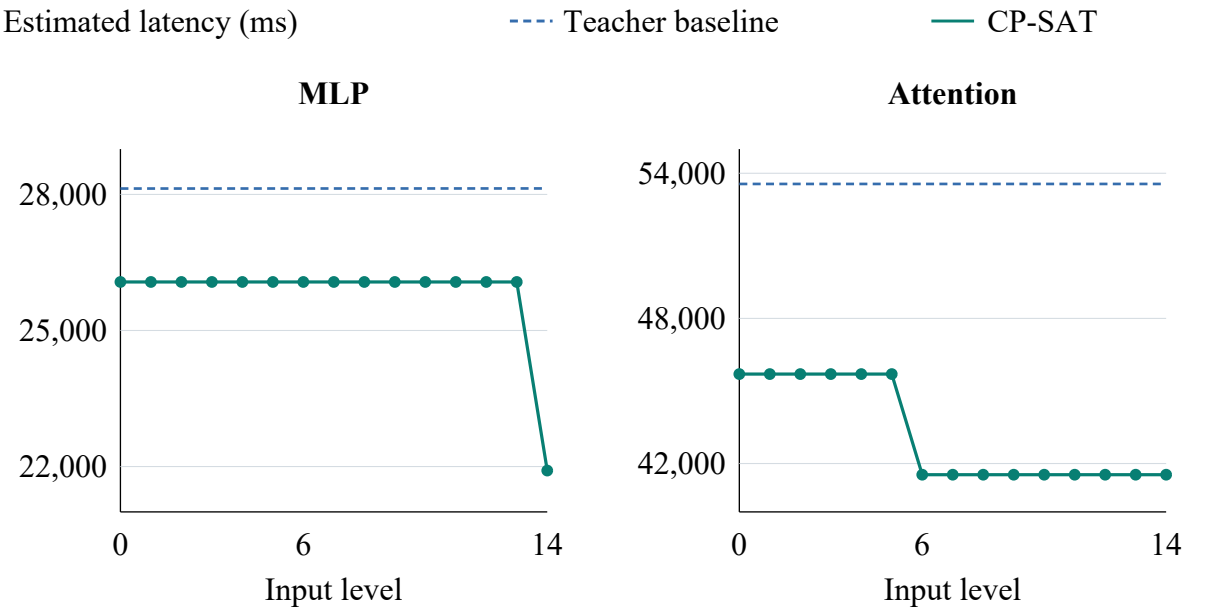


Figure 1. Estimated latency across integer input levels; connecting lines aid reading. The sharp drops occur at MLP input 14 and attention input 6.

The course also reports depth, the maximum original level consumption along a path: 26 for MLP and 62 for attention. Its level budget is 14*B1 + 13*B2 + input level, and utilisation is depth/budget. These describe level use alongside the latency objective; full per-input metrics remain in the saved records.

Solve calls totalled 10.6445 seconds (MLP 0.0075-0.0132, attention 0.5166-0.9656 per condition). Maximum sampled resident set size (RSS, process memory) was 118.36 MiB. Samples every 20 ms include imported libraries and may miss a peak. These observations describe one Windows AMD64 run's solver calls and process, without measuring end-to-end or cross-device performance.

6. Understanding the cost changes

A higher starting level can allow early operations to proceed without a refresh. In the saved attention solution at input 6, the query and key projections each consume two levels and deliver level 4 to the next ciphertext multiplication. Compared with input 0, this avoids an early B1 on four ciphertexts. That removal is the only change in nonzero operation costs between these two solutions and explains the 4160.60 ms drop.

We observe the same kind of change for MLP at input 14. Its saved solution removes an early B1 relative to input 0, saving 4160.60 ms, while a conjugation costs 2.24 ms more. Together these changes explain the net reduction of 4158.36 ms. Each input has its own matching solver bound; the comparison explains the difference between those recorded optima.

The results also show why we minimise full cost. Refreshing changes the levels at which ordinary operators run, so spending more on refreshes can be offset elsewhere. At attention input 0, the course bootstrap count rises from the baseline's 17 to 18, yet estimated latency falls 14.6755%: ordinary-operator costs fall 8900.0554 ms while refresh costs rise 1040.1500 ms. This describes a trade-off between two saved placements, without a causal ablation or a comparison against an optimiser that minimises refresh count.

The saved graphs make these comparisons inspectable. Complete MLP input-14 and attention input-0 diagrams are provided with them. Since equally optimal zero-cost switch arrangements can differ, each diagram refers to its particular saved graph.

7. Scope and remaining work

We have implemented and checked a placement method for two supplied graphs under the course's rules and fitted latency functions. The resulting improvements concern that model. Real fully homomorphic encryption (FHE) speed, behaviour on unseen or larger networks, and runtime rankings against other optimisers would require different evidence. The shared assumptions in our checks also limit their independence.

To complete the project, we will freeze the final method and comparison protocol, run all 30 conditions with formal CPU resource reporting, and provide placement diagrams for at least two input levels per block, checked against saved DAGs. Independent reproduction in a clean environment must then resolve necessary discrepancies and feed corrections into the final report.

The version README links the optimiser, saved results, evaluation and plotting commands, exact execution revision, and report generation inputs. The required public online repository URL and grading revision remain to be supplied; the current local evidence package is the basis for that later course delivery.